

Cours 01 — Les fondements de Git

Module 1/8 — Formation Git & GitHub

Objectifs

À la fin de ce module, vous saurez : - comprendre ce qu'est un **shell** (Bash) et vous déplacer dans un terminal ; - expliquer ce qu'est un système de gestion de versions et pourquoi Git est distribué ; - créer un dépôt local et comprendre le cycle **répertoire de travail** → **index (staging)** → **dépôt** ; - enregistrer des modifications avec `add` et `commit`, et inspecter l'état avec `status`, `log` et `diff`.

Théorie

Avant Git : le shell et Bash

Git est avant tout un **outil en ligne de commande** : on l'utilise dans un terminal, via un **shell** — le programme qui lit vos commandes, les exécute et affiche le résultat.

Bash (*Bourne Again SHell*) est le shell standard du monde Unix/Linux et macOS. Sur Windows, l'installation de Git fournit **Git Bash**, un environnement Bash complet : c'est lui que la documentation Git suppose presque toujours. PowerShell fonctionne aussi pour Git, mais Bash reste la *lingua franca* (exemples en ligne, scripts, GitHub Actions tournent sous Bash).

« Mais j'ai plein de terminaux ! » — [s'y retrouver](#)

Sur une machine Windows typique, on croise plusieurs choses qui *ressemblent* à des terminaux, et c'est facile de s'y perdre. Distinguons deux notions :

- le **terminal** est la *fenêtre* qui affiche du texte ;
- le **shell** est le *programme* qui tourne dedans et interprète vos commandes.

Ce que vous voyez	C'est quoi en réalité ?	Git y fonctionne ?
PowerShell (Windows)	Un shell Microsoft, avec sa propre syntaxe (différente de Bash)	✅ Oui, si Git est installé
Invite de commandes (cmd)	L'ancien shell Windows	✅ Oui, mais syntaxe encore différente
Git Bash	Un terminal + le shell Bash, fournis avec Git	✅ Oui — c'est l'environnement de référence de cette formation
Terminal de PyCharm	Juste une <i>fenêtre</i> : elle héberge un shell (PowerShell par défaut sur Windows, configurable en Git Bash)	✅ Oui — mêmes commandes que le shell hébergé
Anaconda Prompt / conda	Conda n'est pas un shell : c'est un gestionnaire d'environnements Python. L'« Anaconda Prompt » est un cmd/PowerShell avec conda pré-activé	✅ Oui, si Git est installé

À retenir :

1. **Git est le même partout.** `git status` fait exactement la même chose dans Git Bash, PowerShell ou le terminal de PyCharm. Ce qui change, c'est la syntaxe des commandes *autour* de Git (`ls -la`, `touch`, `~`... sont du Bash).
2. **Cette formation utilise la syntaxe Bash.** Si une commande non-Git échoue dans PowerShell, ouvrez Git Bash — tout y fonctionnera tel quel.
3. **PyCharm ne fait que fenêtrer un shell.** Astuce : dans *Settings* → *Tools* → *Terminal*, vous pouvez choisir Git Bash comme shell par défaut, et ainsi suivre la formation directement dans PyCharm.
4. **Conda et Git sont indépendants.** Conda gère vos paquets Python ; Git gère vos versions de fichiers. Vous pouvez utiliser Git dans un environnement conda activé sans conflit.

Notions de base :

Concept	Description
Invite (<i>prompt</i>)	La ligne utilisateur@machine:~/projet\$ qui attend votre commande
Répertoire courant	Le dossier « où vous êtes » ; toutes les commandes s'y rapportent
Chemin relatif / absolu	<code>../donnees</code> (depuis ici) vs <code>/c/Users/Moi/projet</code> (complet)
~	Raccourci vers votre dossier personnel (<i>home</i>)
Tabulation	Complète automatiquement noms de fichiers et commandes — utilisez-la !
Flèche ↑	Rappelle les commandes précédentes

Les commandes Bash essentielles pour suivre cette formation :

```
pwd                # afficher le répertoire courant (print working directory)
ls                 # lister les fichiers ; ls -la pour tout voir (fichiers
cachés inclus)
cd projets/demo-git # se déplacer ; cd .. pour remonter ; cd ~ pour rentrer
chez soi
mkdir demo-git     # créer un dossier
touch notes.md     # créer un fichier vide
echo "bonjour" > notes.md # écrire dans un fichier (> écrase, >> ajoute)
cat notes.md       # afficher le contenu d'un fichier
cp source.md copie.md # copier ; mv pour déplacer/renommer ; rm pour supprimer
clear              # nettoyer l'écran
```

Pourquoi c'est important pour Git : le dossier `.git` est **caché** (`ls -la` le révèle), les chemins passés à `git add` sont relatifs au répertoire courant, et les messages d'erreur de Git supposent que vous savez où vous êtes (`pwd`).

Qu'est-ce que Git ?

Git est un **gestionnaire de versions distribué** : chaque copie du dépôt contient l'historique complet du projet. Contrairement à un système centralisé, on peut committer, consulter l'historique et créer des branches **sans connexion réseau**.

Les trois zones

Zone	Rôle	Commande d'entrée
Répertoire de travail	Vos fichiers tels qu'ils sont sur le disque	(édition normale)
Index (<i>staging area</i>)	La « salle d'attente » du prochain commit	git add
Dépôt (.git/)	L'historique permanent, sous forme de commits	git commit

Un **commit** est une photographie complète du projet à un instant donné, identifiée par un **hash SHA-1** (ex. a3f9c12), avec un auteur, une date et un message.

Au fait : pourquoi parfois - et parfois -- ?

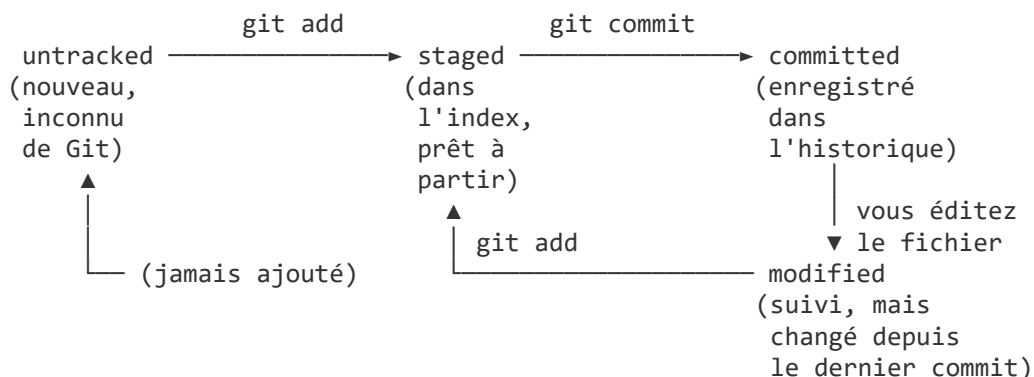
Ce n'est pas une incohérence, c'est une convention Unix que Git respecte :

- - **(un tiret) introduit une option courte** : une seule lettre, rapide à taper. Ex. `git commit -m "message"`, `ls -l`. Les options courtes peuvent se combiner : `ls -la = ls -l -a`.
- -- **(deux tirets) introduit une option longue** : un mot complet, plus lisible. Ex. `git log --oneline`, `git config --global`, `git diff --staged`.

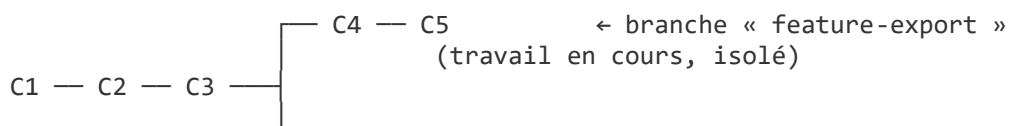
Beaucoup d'options existent sous les deux formes (-m = --message) ; d'autres n'ont qu'une forme longue (--global, --oneline) car une lettre seule serait ambiguë. En pratique : les formes courtes pour le quotidien, les formes longues dans les scripts et la documentation, parce qu'elles se comprennent sans dictionnaire.

Le cycle de vie d'un fichier

Un fichier passe par quatre états, en boucle :



Et côté historique, chaque `git commit` ajoute un nœud au **graphe des commits**. Dès le cours 02 nous créerons des **branches**, c'est-à-dire des lignes de développement parallèles sur ce graphe. Aperçu de ce vers quoi on se dirige :



└─ C6 — C7 ← branche « main »
 (version stable, qui continue d'avancer)

Chaque Cn est un commit (une photographie du projet). Les deux branches divergent à partir de C3 et évoluent indépendamment ; le cours 02 montrera comment les **fusionner**.

Commandes commentées

Configuration initiale — git config

```
git config --global user.name "Prénom Nom"
git config --global user.email "votre.email@exemple.com"
git config --global init.defaultBranch main
```

À faire **une seule fois par machine**. --global signifie que le réglage s'applique à tous vos dépôts (il est stocké dans ~/.gitconfig) ; sans lui, le réglage ne vaudrait que pour le dépôt courant.

Est-ce que je peux mettre n'importe quoi ? Techniquement oui : user.name et user.email ne sont **pas des identifiants de connexion** — Git ne vérifie rien, ce sont de simples étiquettes apposées sur chaque commit pour dire « qui a écrit ça ». Mais en pratique, **utilisez l'adresse e-mail de votre compte GitHub** : c'est grâce à elle que GitHub reconnaît vos commits, les relie à votre profil et colorie votre graphique de contributions. Un e-mail inconnu de GitHub = des commits « anonymes » sur votre propre projet. (Le nom, lui, peut être votre vrai nom ou un pseudo, au choix.)

init.defaultBranch main fait que les nouveaux dépôts nomment leur branche principale main (le standard actuel, aligné sur GitHub) plutôt que l'ancien nom master.

Créer un dépôt — git init

```
git init
```

Transforme le dossier courant en dépôt Git : la commande crée le sous-dossier caché .git/, où Git rangera tout l'historique. Vos fichiers ne sont pas touchés — à ce stade, ils sont tous untracked. À lancer une seule fois, à la racine du projet.

Inspecter l'état — git status

```
git status
```

Le tableau de bord : quels fichiers sont modifiés, lesquels sont dans l'index (prêts à être committés), lesquels sont inconnus de Git. Ne modifie rien ; on peut (et on doit) le lancer sans arrêt.

Préparer un commit — git add

```
git add gestionnaire_ordres.py  # ajoute ce fichier précis à l'index
git add .                      # ajoute tout le dossier courant (récursif)
```

Copie l'état actuel du ou des fichiers dans l'**index** (la « salle d'attente »). Rien n'est encore enregistré dans l'historique : add sert justement à choisir *quoi* mettre dans le prochain commit. git add . est pratique mais aveugle — vérifiez avec git status que vous n'embarquez pas de fichiers indésirables.

Enregistrer — git commit

```
git commit -m "Ajoute le squelette du gestionnaire d'ordres"
```

Prend tout ce qui est dans l'index et en fait un **commit** : une photographie permanente, horodatée et signée (avec le user.name/user.email configurés plus haut). -m fournit le message directement ; sans -m, Git ouvre un éditeur de texte pour le saisir.

Lire l'historique — git log

```
git log                               # liste détaillée : hash complet, auteur, date,
message
git log --oneline --graph             # une ligne par commit + dessin du graphe des
branches
```

--oneline condense chaque commit en hash-court message ; --graph dessine en ASCII les branches et fusions (peu utile aujourd'hui, indispensable dès le cours 02).

Comparer — git diff et git show

```
git diff          # ce que vous avez modifié mais pas encore "add"-é
git diff --staged # ce qui est dans l'index, pas encore committé
git show a3f9c12  # le détail d'un commit précis (via son hash)
```

git diff seul répond à « qu'est-ce que j'ai changé depuis le dernier add ? » ; --staged répond à « qu'est-ce qui partira si je committe maintenant ? ». git show prend un hash (affiché par git log) et montre le message et les modifications de ce commit-là.

Exercices pratiques

0. **Prise en main de Bash.** Dans Git Bash : affichez votre position (pwd), allez dans votre dossier personnel (cd ~), créez un dossier exercices-git, entrez-y, créez un fichier test.txt avec echo, affichez-le avec cat, puis listez le dossier avec ls -la.
1. **Premier dépôt.** Créez un dossier demo-git, initialisez un dépôt, créez un fichier notes.md, puis faites-en votre premier commit. Vérifiez avec ls -la que le dossier caché .git a bien été créé.
2. **Cycle complet.** Modifiez notes.md, observez git status et git diff avant et après git add, puis committez. Vérifiez le résultat avec git log --oneline.
3. **Staging sélectif.** Créez deux fichiers, modifiez-les, mais ne committez que l'un des deux. Vérifiez avec git status que l'autre reste non suivi.
4. **Lecture d'historique.** Faites trois commits, puis affichez uniquement le deuxième avec git show.

Solutions des exercices

Solution 0 — Prise en main de Bash

```
pwd          # affiche par ex. /c/Users/Moi
cd ~         # retour au dossier personnel
mkdir exercices-git # créer le dossier
cd exercices-git # y entrer
echo "premier essai" > test.txt # créer le fichier avec du contenu
cat test.txt  # affiche : premier essai
ls -la       # liste tout, y compris . et ..
```

Solution 1 — Premier dépôt

```
mkdir demo-git
cd demo-git
git init          # crée le dépôt (dossier .git/)
echo "# Mes notes" > notes.md
git add notes.md  # met le fichier dans l'index
git commit -m "Premier commit : ajoute notes.md"
ls -la           # on voit bien le dossier caché .git/
```

Solution 2 — Cycle complet

```
echo "nouvelle ligne" >> notes.md # >> ajoute à la fin (>, lui, écraserait)

git status      # notes.md apparaît en rouge : "modified", non indexé
git diff        # montre la ligne ajoutée (préfixée par +)

git add notes.md
```

```
git status          # notes.md apparaît en vert : "staged", prêt à committer
git diff            # n'affiche plus rien : travail et index sont identiques
git diff --staged # ...c'est ici qu'on voit maintenant le changement
```

```
git commit -m "Ajoute une ligne aux notes"
git log --oneline # deux commits : le premier + celui-ci
```

L'important : `git diff` (travail ↔ index) et `git diff --staged` (index ↔ dernier commit) « se passent le relais » au moment du `git add`.

Solution 3 — Staging sélectif

```
echo "contenu A" > fichier_a.txt
echo "contenu B" > fichier_b.txt
```

```
git status          # les deux apparaissent en "untracked"
```

```
git add fichier_a.txt # on n'indexe QUE le premier
git commit -m "Ajoute fichier_a"
```

```
git status          # fichier_b.txt est toujours "untracked" :
                    # il n'a jamais été add-é, Git l'ignore
```

C'est toute la force de l'index : on choisit précisément ce qui entre dans chaque commit, même si plusieurs fichiers ont changé en même temps.

Solution 4 — Lecture d'historique

```
echo "v1" > journal.txt && git add . && git commit -m "Commit 1"
echo "v2" > journal.txt && git add . && git commit -m "Commit 2"
echo "v3" > journal.txt && git add . && git commit -m "Commit 3"
```

```
git log --oneline
# c3d4e5f Commit 3
# b2c3d4e Commit 2    ← on repère le hash du deuxième commit
# a1b2c3d Commit 1
```

```
git show b2c3d4e      # remplacez par VOTRE hash (git show HEAD~1 marche aussi :
                    # « le commit juste avant le dernier »)
```

Points clés à retenir

- Git se pilote depuis un **shell** ; sous Windows, Git Bash offre l'environnement Bash de référence. Maîtriser `pwd`, `cd`, `ls` -la est un prérequis à tout le reste.
- Git est **distribué** : l'historique complet est local.
- `add` prépare, `commit` enregistre — ce sont deux étapes distinctes et c'est une force.
- `git status` est votre boussole : consultez-la avant chaque commande importante.
- Un bon message de commit décrit le **pourquoi** du changement, pas seulement le quoi.